

Student Lab Guide

Python Programming Course — Module 2

Numerical Computation, Strings, Collections and a Mini Physics Engine

Objective

In this lab you will write and run small Python programs from the command line on your own laptop, using the setup from Module 1. Work through the parts in order. By the end of the lab, every box should be checked.

- ☐ Ten lab programs created in a module2 folder and run with python from the terminal
- ☐ Two reusable modules written: physics.py and report.py
- ☐ Unit tests passing: Ran 9 tests ... OK
- ☐ Your module2 work committed and pushed to GitHub

Note: This lab needs the Module 1 setup: Python 3.13, a PythonCourse folder with an activated .venv, and VS Code with its terminal set to Command Prompt. Only the Python standard library is used — you do not need to install any packages.

Topics Covered

1. Numerical computation
2. Engineering formula evaluation
3. Demonstrating floating-point precision issues
4. String manipulation and formatting
5. Collection operations
6. Creating and accessing nested lists and dictionaries
7. Standard Library: the math module overview
8. Python String Methods reference (docs.python.org)
9. Physics Engine Fundamentals: implementing basic kinematic equations as Python expressions
10. Automated Report Generation: formatting raw scientific data into human-readable summaries using f-strings

Part 1 — Prepare Your Module 2 Folder

Step 1. Open the course folder

Open VS Code. Select File → Open Folder, choose your PythonCourse folder, then Select Folder.

Step 2. Open the terminal and activate the virtual environment

Select Terminal → New Terminal. Check the prompt does not start with PS (if it does, switch to Command Prompt as in Module 1, Part 9). Then enter:

```
.venv\Scripts\activate
```

The prompt should now start with (.venv).

Step 3. Create the module2 folder and move into it

```
mkdir module2  
cd module2
```

Your prompt should now look similar to:

```
(.venv) C:\Users\YourName\Documents\PythonCourse\module2>
```

How every part of this lab works

1. In the VS Code Explorer, select the module2 folder, then New File, and type the file name exactly as shown (including .py).
2. Type the code from this guide into the file and save it (Ctrl + S).
3. Run it in the terminal with python followed by the file name.
4. Compare what you see with the “You should see” box, then do the Exercise.

Tip: Type the code yourself rather than copying it from a chat app or PDF viewer. Chat apps often change straight quotes (") into curly quotes (“ ”), which Python rejects with SyntaxError: invalid character.

Note: Every time you open a new terminal you must activate .venv again and cd into module2. If python says can't open file ... No such file or directory, you are in the wrong folder.

Part 2 — Numerical Computation

Python has three built-in number types: **int** (whole numbers of any size), **float** (decimals with about 15–16 significant digits) and **complex**.

Operator	Meaning	Example → result
+ - *	add, subtract, multiply	7 * 3 → 21
/	true division (always a float)	7 / 2 → 3.5
//	floor division	7 // 2 → 3
%	remainder (modulo)	7 % 2 → 1
**	power	2 ** 10 → 1024

Step 1. Create 01_numbers.py

In the VS Code Explorer, inside the module2 folder, create a new file named **01_numbers.py** and type:

```
# Module 2 - Part 2: Numerical computation  
a = 17  
b = 5
```

```

print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)          # true division: always a float
print("a // b =", a // b)       # floor division
print("a % b =", a % b)         # remainder
print("a ** b =", a ** b)       # power
print("divmod(a, b) =", divmod(a, b))

print()
print("-7 // 2 =", -7 // 2)      # floors towards minus infinity
print("-2 ** 2 =", -2 ** 2)     # ** is done before the minus sign
print("(-2) ** 2 =", (-2) ** 2)
print("2 ** 100 =", 2 ** 100)   # int has no size limit
print("1e308 * 10 =", 1e308 * 10) # float does
print("type(3) =", type(3), " type(3.0) =", type(3.0))
print("int(7.9) =", int(7.9))
print("round(7.5) =", round(7.5), " round(8.5) =", round(8.5))

```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 01_numbers.py
```

You should see:

```

a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
a % b = 2
a ** b = 1419857
divmod(a, b) = (3, 2)

-7 // 2 = -4
-2 ** 2 = -4
(-2) ** 2 = 4
2 ** 100 = 1267650600228229401496703205376
1e308 * 10 = inf
type(3) = <class 'int'>  type(3.0) = <class 'float'>
int(7.9) = 7
round(7.5) = 8  round(8.5) = 8

```

What to notice

- `-7 // 2` is `-4`, not `-3`: floor division always rounds down.
- `-2 ** 2` is `-4` because `**` is calculated before the minus sign. Use brackets when in doubt.
- `round()` rounds halves to the nearest even number, so `round(8.5)` is `8`.

Exercise 2

A rope 1000 cm long is cut into pieces of 37 cm. In a new file `ex02.py`, use `//` and `%` to print how many full pieces you get and how many centimetres are left over.

Part 3 — Engineering Formula Evaluation

An engineering formula becomes a Python expression almost word for word. Good habits:

- one variable per quantity, with a descriptive name;
- the unit written in a comment next to each input;
- everything in SI units, converting only when you print.

Step 1. Create 02_engineering.py

In the VS Code Explorer, inside the module2 folder, create a new file named **02_engineering.py** and type:

```
# Module 2 - Part 3: Engineering formula evaluation

# Ohm's law ( $V = I * R$ ) and electrical power ( $P = V * I$ )
voltage = 230.0      # V
resistance = 52.9     # ohm
current = voltage / resistance      # A
power = voltage * current           # W
print(f"Current = {current:.3f} A, Power = {power:.1f} W")

# Reynolds number:  $Re = \text{density} * \text{velocity} * \text{diameter} / \text{viscosity}$ 
density = 998.0       # kg/m^3 (water at 20 C)
velocity = 1.5        # m/s
diameter = 0.05       # m
viscosity = 1.002e-3  # Pa*s
reynolds = density * velocity * diameter / viscosity
if reynolds < 2300:
    regime = "laminar"
elif reynolds > 4000:
    regime = "turbulent"
else:
    regime = "transitional"
print(f"Re = {reynolds:,.0f} -> {regime} flow")

# Bending stress in a rectangular beam:  $\text{stress} = M * y / I$ , with  $I = b * h^3 / 12$ 
width = 0.10          # m
height = 0.20         # m
moment = 15_000       # N*m
second_moment = width * height ** 3 / 12      # m^4
y_max = height / 2    # m
stress = moment * y_max / second_moment       # Pa
print(f"I = {second_moment:.3e} m^4, max stress = {stress / 1e6:.2f} MPa")
```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 02_engineering.py
```

You should see:

```
Current = 4.348 A, Power = 1000.0 W
Re = 74,701 -> turbulent flow
I = 6.667e-05 m^4, max stress = 22.50 MPa
```

What to notice

- {current:.3f} prints 3 decimal places; {reynolds:,.0f} adds a thousands separator; {second_moment:.3e} uses scientific notation.
- 15_000 is the same number as 15000 — underscores just make large numbers easier to read.
- if / elif / else chooses the flow regime from the calculated value.

Exercise 3

The kinetic energy of a moving body is $E = \frac{1}{2} m v^2$. In ex03.py, compute it for a 1200 kg car travelling at 90 km/h (convert to m/s first) and print the answer in kJ with one decimal place.

Part 4 — Demonstrating Floating-Point Precision Issues

Computers store floats in binary. Most decimal fractions, such as 0.1, have no exact binary form, so a tiny rounding error is stored with them. This is not a Python bug — every language that uses standard floating point behaves the same way.

Step 1. Create 03_floats.py

In the VS Code Explorer, inside the module2 folder, create a new file named **03_floats.py** and type:

```
# Module 2 - Part 4: Floating-point precision
import math
from decimal import Decimal
from fractions import Fraction

print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3 ->", 0.1 + 0.2 == 0.3)
print(f"0.1 is stored as {0.1:.20f}")
print("math.isclose(0.1 + 0.2, 0.3) ->", math.isclose(0.1 + 0.2, 0.3))

total = 0.0
for _ in range(10):
    total += 0.1
print("0.1 added ten times =", total)
print("math.fsum of ten 0.1 =", math.fsum([0.1] * 10))

print("round(2.675, 2) =", round(2.675, 2))
print(f"2.675 is stored as {2.675:.20f}")
print("(1e16 + 1) - 1e16 =", (1e16 + 1) - 1e16)

print("Decimal:", Decimal("0.1") + Decimal("0.2"))
print("Fraction:", Fraction(1, 10) + Fraction(2, 10))
```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 03_floats.py
```

You should see:

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 -> False
0.1 is stored as 0.1000000000000000555
math.isclose(0.1 + 0.2, 0.3) -> True
0.1 added ten times = 0.9999999999999999
math.fsum of ten 0.1 = 1.0
round(2.675, 2) = 2.67
2.675 is stored as 2.67499999999999982236
(1e16 + 1) - 1e16 = 0.0
Decimal: 0.3
Fraction: 3/10
```

What to notice

- Never compare floats with ==. Use math.isclose() instead.
- Small errors add up in long sums; math.fsum() adds them accurately.
- round(2.675, 2) gives 2.67 because 2.675 is really stored as 2.67499999...
- Use Decimal when exact decimal results matter, for example money.

Exercise 4

Predict, then check in ex04.py: is 0.1 * 3 == 0.3 True or False? What does math.isclose(0.1 * 3, 0.3) return?

Part 5 — String Manipulation and Formatting

Strings are immutable: every string method returns a new string and leaves the original unchanged. A common task is cleaning a messy line of raw data, then printing it neatly with an f-string.

Step 1. Create 04_strings.py

In the VS Code Explorer, inside the module2 folder, create a new file named **04_strings.py** and type:

```
# Module 2 - Part 5: String manipulation and formatting
raw = "    T-101 , reactor TEMPERATURE , 351.2749 , kelvin  "

parts = [p.strip() for p in raw.split(",")]
print(parts)

tag, name, value, unit = parts
name = name.title()
value = float(value)
print(tag, "|", name, "|", value, "|", unit.upper())

print(name.lower())
print(name.replace("Reactor", "Boiler"))
```

```

print("find('Temp') ->", name.find("Temp"))
print("startswith('Reactor') ->", name.startswith("Reactor"))
print("first char:", tag[0], " last 3:", tag[-3:], " reversed:", name[::-1])
print(" | ".join(parts))

print()
print(f"{name}: {value:.2f} K")
print(f"[{tag:<10}] [{tag:>10}] [{tag:^10}]")
print(f"{1234567.891:,.2f}")
print(f"{0.000123:.2e}")
print(f"{0.4567:.1%}")
print(f"{42:05d}")
print(f"{value=}")

```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 04_strings.py
```

You should see:

```

['T-101', 'reactor TEMPERATURE', '351.2749', 'kelvin']
T-101 | Reactor Temperature | 351.2749 | KELVIN
reactor temperature
Boiler Temperature
find('Temp') -> 8
startswith('Reactor') -> True
first char: T last 3: 101 reversed: erutarepmeT rotcaeR
T-101 | reactor TEMPERATURE | 351.2749 | kelvin

Reactor Temperature: 351.27 K
[T-101      ] [      T-101] [  T-101   ]
1,234,567.89
1.23e-04
45.7%
00042
value=351.2749

```

f-string format specifiers used above

Specifier	Meaning	Output
:.2f	2 decimal places	351.27
:<10 :>10 :^10	left, right, centre in 10 characters	[T-101]
:,.2f	thousands separator	1,234,567.89
:.2e	scientific notation	1.23e-04
:.1%	percentage	45.7%
:05d	zero-padded integer	00042
{value=}	name and value (debugging)	value=351.2749

Exercise 5

In ex05.py, clean the line " p-07 ; inlet pressure;101.337 ;KPA " (note the ; separators) and print it as:

```
P-07 Inlet Pressure = 101.34 kPa
```

Part 6 — Collection Operations

Type	Written as	Ordered	Changeable	Duplicates
list	[1, 2, 3]	yes	yes	yes
tuple	(1, 2, 3)	yes	no	yes
set	{1, 2, 3}	no	yes	no
dict	{"a": 1}	yes (insertion)	yes	keys unique

Step 1. Create 05_collections.py

In the VS Code Explorer, inside the module2 folder, create a new file named **05_collections.py** and type:

```
# Module 2 - Part 6: Collection operations
readings = [23.1, 22.8, 24.5, 23.9]
readings.append(25.2)
readings.extend([21.7, 22.0])
print("readings:", readings)
print("count:", len(readings), " first:", readings[0], " last:", readings[-1])
print("slice [1:4]:", readings[1:4])
print("sorted:", sorted(readings))
print("min:", min(readings), " max:", max(readings))
print("mean:", round(sum(readings) / len(readings), 2))
print("above 23:", [r for r in readings if r > 23])

point = (3.0, 4.0)
x, y = point
print("tuple unpacked: x =", x, " y =", y)

group_a = {"S1", "S2", "S3"}
group_b = {"S3", "S4"}
print("union:", sorted(group_a | group_b))
print("intersection:", sorted(group_a & group_b))
print("difference:", sorted(group_a - group_b))

constants = {"g": 9.81, "c": 299_792_458}
constants["h"] = 6.626e-34
print("g =", constants["g"])
print("k =", constants.get("k", "not defined"))
for key, val in constants.items():
    print(" ", key, "=", val)
```

```
quantities = ["mass", "speed", "time"]
values = [2.0, 3.5, 10]
print(dict(zip(quantities, values)))
for i, q in enumerate(quantities, start=1):
    print(i, q)
```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 05_collections.py
```

You should see:

```
readings: [23.1, 22.8, 24.5, 23.9, 25.2, 21.7, 22.0]
count: 7 first: 23.1 last: 22.0
slice [1:4]: [22.8, 24.5, 23.9]
sorted: [21.7, 22.0, 22.8, 23.1, 23.9, 24.5, 25.2]
min: 21.7 max: 25.2
mean: 23.31
above 23: [23.1, 24.5, 23.9, 25.2]
tuple unpacked: x = 3.0 y = 4.0
union: ['S1', 'S2', 'S3', 'S4']
intersection: ['S3']
difference: ['S1', 'S2']
g = 9.81
k = not defined
g = 9.81
c = 299792458
h = 6.626e-34
{'mass': 2.0, 'speed': 3.5, 'time': 10}
1 mass
2 speed
3 time
```

What to notice

- A list comprehension such as `[r for r in readings if r > 23]` builds a new list in one line.
- Sets have no order, so the program uses `sorted()` to print them the same way every time.
- `dict.get(key, default)` returns the default instead of crashing when the key is missing.

Exercise 6

In `ex06.py`, use a list comprehension to convert every value in `readings` from °C to °F ($F = C \times 9/5 + 32$), rounded to one decimal place, and print the new list.

Part 7 — Creating and Accessing Nested Lists and Dictionaries

Collections can contain other collections. To reach an inner value, chain the indexes or keys from left to right: `experiments["EXP-01"]["temps_C"][-1]` means “the EXP-01 entry, then its `temps_C` list, then the last item”.

Step 1. Create 06_nested.py

In the VS Code Explorer, inside the module2 folder, create a new file named **06_nested.py** and type:

```
# Module 2 - Part 7: Nested lists and dictionaries
import json

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]
print("row 1:", matrix[1])
print("row 1, column 2:", matrix[1][2])
print("first column:", [row[0] for row in matrix])
print("transpose:", [list(col) for col in zip(*matrix)])

experiments = {
    "EXP-01": {"material": "steel", "temps_C": [20, 150, 300], "passed": True},
    "EXP-02": {"material": "aluminium", "temps_C": [20, 90], "passed": False},
}
print(experiments["EXP-01"]["material"])
print(experiments["EXP-01"]["temps_C"][-1])

experiments["EXP-02"]["temps_C"].append(120)
experiments["EXP-03"] = {"material": "copper", "temps_C": [20, 60], "passed": True}

for exp_id, info in experiments.items():
    print(f"{exp_id}: {info['material']}<10} max {max(info['temps_C']):>3} C
    passed={info['passed']}")

print(json.dumps(experiments["EXP-03"], indent=2))

grid = [[0] * 3] * 3          # trap: the same inner list three times
grid[0][0] = 1
print("trap: ", grid)
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
print("correct:", grid)
```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 06_nested.py
```

You should see:

```
row 1: [4, 5, 6]
row 1, column 2: 6
first column: [1, 4, 7]
transpose: [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
steel
300
EXP-01: steel      max 300 C  passed=True
EXP-02: aluminium max 120 C  passed=False
```

```
EXP-03: copper      max  60 C  passed=True
{
  "material": "copper",
  "temps_C": [
    20,
    60
  ],
  "passed": true
}
trap:    [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
correct: [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

What to notice

- `zip(*matrix)` turns rows into columns (a transpose).
- `json.dumps(..., indent=2)` prints any nested structure in a readable layout.
- `[[0] * 3] * 3` repeats the same inner list three times, so changing one row changes all of them. Build grids with a comprehension instead.

Exercise 7

In `ex07.py`, copy the experiments dictionary, add `"operator": "your name"` inside `EXP-01`, then print the average temperature of every experiment that passed.

Part 8 — Standard Library: the math Module

`import math` gives you mathematical constants and functions that are fast and accurate. Full reference: [math — Mathematical functions](#).

Group	Examples
Constants	<code>pi</code> , <code>e</code> , <code>tau</code> , <code>inf</code> , <code>nan</code>
Powers and roots	<code>sqrt</code> , <code>pow</code> , <code>exp</code> , <code>hypot</code> , <code>cbrt</code>
Logarithms	<code>log</code> , <code>log10</code> , <code>log2</code>
Trigonometry	<code>sin</code> , <code>cos</code> , <code>tan</code> , <code>atan2</code> , <code>radians</code> , <code>degrees</code>
Rounding	<code>floor</code> , <code>ceil</code> , <code>trunc</code>
Integers	<code>factorial</code> , <code>comb</code> , <code>perm</code> , <code>gcd</code>
Float checks	<code>isclose</code> , <code>isnan</code> , <code>isinf</code> , <code>fsum</code>

Step 1. Create `07_math_module.py`

In the VS Code Explorer, inside the `module2` folder, create a new file named `07_math_module.py` and type:

```
# Module 2 - Part 8: The math module
import math

print("pi =", math.pi)
```

```

print("e =", math.e)
print("sqrt(2) =", math.sqrt(2))
print("hypot(3, 4) =", math.hypot(3, 4))
print("sin(30 deg) =", math.sin(math.radians(30)))
print("45 deg in radians =", math.radians(45))
print("log10(1000) =", math.log10(1000))
print("log(8, 2) =", math.log(8, 2))
print("floor(-2.5) =", math.floor(-2.5), " ceil(-2.5) =", math.ceil(-2.5))
print("factorial(5) =", math.factorial(5))
print("comb(5, 2) =", math.comb(5, 2))
print("gcd(12, 18) =", math.gcd(12, 18))

names = [n for n in dir(math) if not n.startswith("_")]
print(len(names), "names in math, for example:", names[:8])

```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 07_math_module.py
```

You should see:

```

pi = 3.141592653589793
e = 2.718281828459045
sqrt(2) = 1.4142135623730951
hypot(3, 4) = 5.0
sin(30 deg) = 0.49999999999999994
45 deg in radians = 0.7853981633974483
log10(1000) = 3.0
log(8, 2) = 3.0
floor(-2.5) = -3  ceil(-2.5) = -2
factorial(5) = 120
comb(5, 2) = 10
gcd(12, 18) = 6
62 names in math, for example: ['acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2',
'atanh', 'cbrt']

```

Note: Trigonometric functions work in radians. Convert degrees with `math.radians()` first — that is why `sin(30 deg)` is written `math.sin(math.radians(30))`.

Exercise 8

In `ex08.py`, print `math.atan2.__doc__` to read its documentation, then print the angle in degrees of the vector `x = -1, y = 1`.

Part 9 — Python String Methods Reference

The official list of every string method is in the Python documentation: [String Methods — docs.python.org](https://docs.python.org/3/library/string.html). The ones you will use most:

Method	What it does	Example → result
strip()	remove surrounding spaces	" hi ".strip() → "hi"
lower() / upper()	change case	"Hi".upper() → "HI"
title()	capitalise each word	"flow rate".title() → "Flow Rate"
split(sep)	text → list	"a,b".split(",") → ["a", "b"]
join(list)	list → text	"-".join(["a", "b"]) → "a-b"
replace(old, new)	substitute text	"1,5".replace(",", ".") → "1.5"
find(sub)	position, or -1	"abc".find("c") → 2
startswith / endswith	test the start or end	"data.csv".endswith(".csv") → True
isdigit()	only digits?	"42".isdigit() → True
zfill(n)	pad with zeros	"7".zfill(3) → "007"
center(n, ch)	pad to a width	"ok".center(6, "*") → "ok"
count(sub)	number of occurrences	"banana".count("a") → 3

Step 1. Create 08_string_methods.py

In the VS Code Explorer, inside the module2 folder, create a new file named **08_string_methods.py** and type:

```
# Module 2 - Part 9: Python string methods reference
methods = [m for m in dir(str) if not m.startswith("_")]
print(len(methods), "string methods")
print(methods[:12])

print("' hi '.strip() ->", repr(" hi ".strip()))
print("'flow rate'.title() ->", "flow rate".title())
print("'a,b,c'.split(',') ->", "a,b,c".split(","))
print("'-.join(['a', 'b']) ->", "-".join(["a", "b"]))
print("'1,5'.replace(',', '.') ->", "1,5".replace(",", "."))
print("'data.csv'.endswith('.csv') ->", "data.csv".endswith(".csv"))
print("'42'.isdigit() ->", "42".isdigit())
print("'7'.zfill(3) ->", "7".zfill(3))
print("'ok'.center(6, '*') ->", "ok".center(6, "*"))
print("'banana'.count('a') ->", "banana".count("a"))
```

Save the file (Ctrl + S).

Step 2. Run it

In the terminal (the prompt must start with (.venv) and end in module2>), enter:

```
python 08_string_methods.py
```

You should see:

```
47 string methods
['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs',
'find', 'format', 'format_map', 'index', 'isalnum']
```

```
' hi '.strip() -> 'hi'
'flow rate'.title() -> Flow Rate
'a,b,c'.split(',') -> ['a', 'b', 'c']
'-.'.join(['a', 'b']) -> a-b
'1,5'.replace(',', '.') -> 1.5
'data.csv'.endswith('.csv') -> True
'42'.isdigit() -> True
'7'.zfill(3) -> 007
'ok'.center(6, '*') -> **ok**
'banana'.count('a') -> 3
```

Exercise 9

Pick two string methods that are not in the table above (for example partition or removeprefix), look them up on docs.python.org, and write a one-line example of each in ex09.py.

Part 10 — Physics Engine Fundamentals: Kinematic Equations

For motion with constant acceleration a , starting velocity u , time t and displacement s :

Equation	Python expression
$v = u + at$	<code>u + a * t</code>
$s = ut + \frac{1}{2}at^2$	<code>u * t + 0.5 * a * t ** 2</code>
$v^2 = u^2 + 2as$	<code>math.sqrt(u ** 2 + 2 * a * s)</code>

You will put these formulas in a module — a .py file of functions that other programs can import — so they can be reused and tested.

Step 1. Create physics.py

In the module2 folder create **physics.py** and type:

```
"""Basic kinematics for constant acceleration (SI units: m, s, m/s, m/s^2)."""
import math

G = 9.81 # gravitational acceleration, m/s^2

def final_velocity(u, a, t):
    """v = u + a*t"""
    return u + a * t

def displacement(u, a, t):
    """s = u*t + 1/2*a*t^2"""
    return u * t + 0.5 * a * t ** 2

def velocity_from_displacement(u, a, s):
    """v = sqrt(u^2 + 2*a*s)"""
    v_squared = u ** 2 + 2 * a * s
```

```

if v_squared < 0:
    raise ValueError("u**2 + 2*a*s is negative: the object never reaches s")
return math.sqrt(v_squared)

def projectile(speed, angle_deg, g=G):
    """Launch from ground level; return time of flight, max height and range."""
    theta = math.radians(angle_deg)
    vx = speed * math.cos(theta)
    vy = speed * math.sin(theta)
    time_of_flight = 2 * vy / g
    return {
        "time_of_flight": time_of_flight,
        "max_height": vy ** 2 / (2 * g),
        "range": vx * time_of_flight,
    }

def simulate_fall(height, dt=0.01, g=G):
    """Step a dropped object forward in small time steps until it lands.

    Returns (time, speed). This is how a physics engine works.
    """
    t, y, v = 0.0, height, 0.0
    while y > 0:
        v += g * dt
        y -= v * dt
        t += dt
    return t, v

```

Save the file. physics.py does nothing on its own when run; it only defines functions.

Step 2. Create 09_kinematics.py

Create **09_kinematics.py** in the same folder. import physics loads the module you just wrote:

```

# Module 2 - Part 10: Physics engine fundamentals
import math

import physics

t = 3.0
v = physics.final_velocity(0, physics.G, t)
s = physics.displacement(0, physics.G, t)
print(f"Free fall for {t} s: v = {v:.2f} m/s, distance = {s:.2f} m")
print(f"Speed after falling 20 m: {physics.velocity_from_displacement(0, physics.G, 20):.2f} m/s")

u, a = 25.0, -6.0                # braking car
stop_time = -u / a
print(f"Braking: stops after {stop_time:.2f} s and {physics.displacement(u, a, stop_time):.1f} m")

print()
print("Projectile launched at 20 m/s")
print(f"{'Angle':>5} | {'Flight (s)':>10} | {'Height (m)':>10} | {'Range (m)':>9}")
for angle in range(15, 90, 15):
    r = physics.projectile(20, angle)

```

```

    print(f"{angle:>5} | {r['time_of_flight']:>10.2f} | {r['max_height']:>10.2f} | {r['range']:>9.2f}")

print()
exact = math.sqrt(2 * 20 / physics.G)
for dt in (0.1, 0.01, 0.001):
    t_sim, v_sim = physics.simulate_fall(20, dt)
    print(f"dt = {dt:<6} -> t = {t_sim:.4f} s (exact {exact:.4f} s, error {abs(t_sim - exact):.4f} s)")

```

Step 3. Run it

```
python 09_kinematics.py
```

You should see:

```

Free fall for 3.0 s: v = 29.43 m/s, distance = 44.15 m
Speed after falling 20 m: 19.81 m/s
Braking: stops after 4.17 s and 52.1 m

Projectile launched at 20 m/s
Angle | Flight (s) | Height (m) | Range (m)
15 | 1.06 | 1.37 | 20.39
30 | 2.04 | 5.10 | 35.31
45 | 2.88 | 10.19 | 40.77
60 | 3.53 | 15.29 | 35.31
75 | 3.94 | 19.02 | 20.39

dt = 0.1    -> t = 2.0000 s (exact 2.0193 s, error 0.0193 s)
dt = 0.01   -> t = 2.0200 s (exact 2.0193 s, error 0.0007 s)
dt = 0.001  -> t = 2.0190 s (exact 2.0193 s, error 0.0003 s)

```

What to notice

- A real physics engine (games, simulators) does not use the closed-form formulas. It moves the world forward in tiny time steps dt — `simulate_fall()` does exactly that.
- Smaller dt gives an answer closer to the exact formula, but needs more steps (more computation).
- 45° gives the longest range; 30° and 60° give the same range.

Note: `physics.py` must be in the same folder as `09_kinematics.py` and must be named exactly `physics.py`. Otherwise you will see `ModuleNotFoundError: No module named 'physics'`.

Exercise 10

In `ex10.py`, import `physics` and compare the range of a 20 m/s launch at 45° on Earth with the same launch on the Moon ($g = 1.62 \text{ m/s}^2$).

Part 11 — Automated Report Generation with f-strings

Raw data is just lists of numbers. A report turns it into a summary a person can read: aligned columns, the same number of decimal places, units, and a conclusion. `report.py` does this using only f-strings.

Step 1. Create report.py

```
"""Turn raw scientific measurements into a readable text report."""
import math

def summarize(values):
    """Return count, mean, min, max and (population) standard deviation."""
    n = len(values)
    if n == 0:
        raise ValueError("no values to summarize")
    mean = sum(values) / n
    variance = sum((x - mean) ** 2 for x in values) / n
    return {
        "n": n,
        "mean": mean,
        "min": min(values),
        "max": max(values),
        "std": math.sqrt(variance),
    }

def format_report(title, datasets, decimals=2):
    """datasets maps a quantity name to {"unit": str, "values": [numbers]}."""
    header =
    f"""Quantity:<14>{Unit:<7>{n:>3}{Mean:>10}{Min:>10}{Max:>10}{Std:>8}"""
    rule = "-" * len(header)
    lines = [title, "=" * len(title), header, rule]
    for name, data in datasets.items():
        s = summarize(data["values"])
        lines.append(
            f"""{name:<14}>{data['unit']:<7}>{s['n']:>3}"""
            f"""{s['mean']:>10.{decimals}f}{s['min']:>10.{decimals}f}"""
            f"""{s['max']:>10.{decimals}f}{s['std']:>8.{decimals}f}"""
        )
    total = sum(len(d["values"]) for d in datasets.values())
    lines.append(rule)
    lines.append(f"""{len(datasets)} quantities, {total} readings in total.""")
    return "\n".join(lines)
```

Step 2. Create 10_report.py

```
# Module 2 - Part 11: Automated report generation
import report

raw_data = {
    "Temperature": {"unit": "C", "values": [21.43, 21.97, 22.61, 23.08, 22.40]},
    "Pressure": {"unit": "kPa", "values": [101.3, 101.1, 100.9, 101.4, 101.2]},
    "Flow rate": {"unit": "L/min", "values": [12.5, 12.9, 13.4, 12.2, 12.8]},
}

text = report.format_report("Cooling Loop Test - Run 7", raw_data)
print(text)

temps = raw_data["Temperature"]["values"]
s = report.summarize(temps)
conclusion = (
```

```

    f"Temperature averaged {s['mean']:.1f} C (range {s['min']:.1f}-{s['max']:.1f} C) "
    f"and rose by {temps[-1] - temps[0]:+.2f} C over the run."
)
print()
print(conclusion)

with open("report.txt", "w") as f:
    f.write(text + "\n\n" + conclusion + "\n")
print("Saved report.txt")

```

Step 3. Run it

```
python 10_report.py
```

You should see:

```

Cooling Loop Test - Run 7
=====
Quantity      Unit      n      Mean      Min      Max      Std
-----
Temperature    C          5      22.30      21.43      23.08      0.56
Pressure       kPa          5     101.18     100.90     101.40      0.17
Flow rate      L/min          5      12.76      12.20      13.40      0.40
-----
3 quantities, 15 readings in total.

Temperature averaged 22.3 C (range 21.4-23.1 C) and rose by +0.97 C over the run.
Saved report.txt

```

The program also saved the report to a file. Display it with:

```
type report.txt
```

What to notice

- {name:<14} left-aligns text in 14 characters and {s['mean']:>10.2f} right-aligns a number in 10 characters with 2 decimals — that is what makes the columns line up.
- {...:+.2f} always shows the sign, so a rise prints as +0.97.
- decimals is a parameter, so the same function can produce reports with any precision.

Exercise 11

In ex11.py, call `report.format_report` with `decimals=3`, and add a fourth quantity of your own (for example "Voltage" in V with five readings).

Part 12 — Check Your Work with Unit Tests

A unit test is code that checks other code automatically. `test_module2.py` checks the functions in `physics.py` and `report.py`. All tests must pass before you submit.

Step 1. Create test_module2.py

```
import math
import unittest

from physics import (G, displacement, final_velocity, projectile,
                    simulate_fall, velocity_from_displacement)
from report import format_report, summarize

class TestKinematics(unittest.TestCase):
    def test_final_velocity(self):
        self.assertAlmostEqual(final_velocity(0, 9.81, 2), 19.62)

    def test_displacement(self):
        self.assertAlmostEqual(displacement(5, 2, 3), 24.0)

    def test_velocity_from_displacement(self):
        self.assertAlmostEqual(velocity_from_displacement(0, 9.81, 20),
math.sqrt(392.4))

    def test_unreachable_height_raises(self):
        with self.assertRaises(ValueError):
            velocity_from_displacement(1, -9.81, 10)

    def test_projectile_range_at_45_degrees(self):
        self.assertAlmostEqual(projectile(20, 45)["range"], 20 ** 2 / G)

    def test_simulation_matches_formula(self):
        t, _ = simulate_fall(20, dt=0.001)
        self.assertAlmostEqual(t, math.sqrt(2 * 20 / G), delta=0.01)

class TestReport(unittest.TestCase):
    def test_summarize(self):
        s = summarize([1.0, 2.0, 3.0])
        self.assertEqual(s["n"], 3)
        self.assertAlmostEqual(s["mean"], 2.0)
        self.assertAlmostEqual(s["std"], math.sqrt(2 / 3))

    def test_summarize_empty_raises(self):
        with self.assertRaises(ValueError):
            summarize([])

    def test_format_report(self):
        text = format_report("Run 1", {"Temperature": {"unit": "C", "values": [20.0,
22.5]}})
        self.assertIn("Run 1", text)
        self.assertIn("21.25", text)
        self.assertIn("2 readings", text)

if __name__ == "__main__":
    unittest.main()
```

Step 2. Run the tests

```
python -m unittest -v test_module2
```

You should see nine lines ending in ok, followed by:

```
-----  
Ran 9 tests in 0.001s
```

```
OK
```

Note: The time (0.001s) may differ slightly on your laptop. If a test prints FAIL or ERROR, read the message, fix the function it names, save, and run the tests again.

Part 13 — Save Your Work to GitHub

Go back up to the PythonCourse folder, which is your Git repository from Module 1:

```
cd ..
```

Python creates a `__pycache__` folder when you import your own modules. Like `.env`, it should not be uploaded. Open `.gitignore` in VS Code and add a second line so it reads:

```
.env/  
__pycache__/
```

Then commit and push your Module 2 work:

```
git status  
git add .  
git commit -m "Module 2 lab: numerical computation"  
git push
```

git status before git add should list the module2 folder and `.gitignore`, but not `__pycache__` or `.env`.

Note: If your instructor gave you a separate course repository in Module 1 (Part 15), copy the module2 folder into that repository's folder and run the same commands there.

Part 14 — Final Verification Checklist

Programs

- ☐ 01_numbers.py to 08_string_methods.py run and match the “You should see” boxes.
- ☐ 09_kinematics.py prints the projectile table and the three dt lines.
- ☐ 10_report.py prints the report and creates report.txt.

Tests

- ☐ `python -m unittest -v test_module2` ends with Ran 9 tests ... OK

Exercises

- ☐ Exercises 2 to 11 are saved as ex02.py ... ex11.py in module2 and each one runs.

Understanding

- ☐ You can explain why $0.1 + 0.2$ is not exactly 0.3 and how to compare floats.
- ☐ You can read a value out of a nested dictionary.
- ☐ You can format a number with a fixed width and number of decimals in an f-string.

GitHub

- ☐ Your module2 folder is pushed to GitHub, without .venv or __pycache__.

Important Rules for This Lab

- Run every program from the terminal with python, with (.venv) active and the prompt in module2.
- Type the code yourself; do not paste from chat apps.
- Use only the Python standard library — do not install packages for this lab.
- Keep file names exactly as shown; Python imports depend on them.
- If you get an error, read the last line of the message first — it usually names the problem — and show the full message to the instructor if you are stuck.
- Commit and push your work at the end of each lab session.

Official References

- [Floating-Point Arithmetic: Issues and Limitations — Python Tutorial](#)
- [math — Mathematical functions](#)
- [String Methods](#)
- [Format Specification Mini-Language](#)
- [Data Structures — Python Tutorial](#)
- [Modules — Python Tutorial](#)
- [unittest — Unit testing framework](#)
- [decimal — Decimal fixed-point and floating-point arithmetic](#)

Draft v0.1 for instructor authoring and review (Content Development Workflow, Step 4). Every program and expected output was run on Windows 10 with Python 3.13.15 in a .venv from Command Prompt on 26 Sep 2026.